

Characterization of Lightweight CNN Architectures with INT8 Quantization for Poultry Fecal Disease Detection on Edge Devices

Fatih Jawwad Al Mumtaz

Program Studi Teknik Informatika

Universitas Darussalam Gontor

Email: fatihalmumtaz76@student.cs.unida.gontor.ac.id

Abstract—Poultry fecal disease detection using deep learning has demonstrated promising accuracy, yet existing approaches predominantly rely on cloud-based inference with heavy architectures that are impractical for resource-constrained farm environments. This paper presents a systematic characterization of three lightweight convolutional neural network architectures—MobileNetV2, ShuffleNetV2, and EfficientNet-B0—for classifying poultry fecal images into four disease categories: Coccidiosis, Salmonellosis, Newcastle Disease, and Healthy. Experiments use the Machuve *et al.* dataset as distributed via HuggingFace `Diano/poultry-fecal-f1` (8,770 images: 5,614 train / 1,402 val / 1,754 test, augmented from the original Zenodo 6,812). We evaluate each architecture under post-training INT8 dynamic quantization (ONNX Runtime) on an Intel i5-12400F CPU (200 runs, 10 warmup, CPUExecutionProvider) and on a Raspberry Pi 5 (Broadcom BCM2712, 4× Cortex-A76, Debian 13, ONNX Runtime 1.29.0, 96 thread/batch configurations). All three architectures achieve over 97% FP32 test accuracy through ImageNet transfer learning, with EfficientNet-B0 highest at 98.18% and MobileNetV2 best latency-accuracy trade-off (1.85 ms FP32, 540.8 FPS on x86). Critically, INT8 PTQ introduces 6.4–30.1× latency overhead on x86 CPU and 1.9–2.8× on RPi5 due to dequantization costs, and INT8 thread scaling stalls at 1.1–1.2× (1→4 threads) versus 2.3× for FP32, revealing poor ARM INT8 parallelization. Accuracy exhibits architecture-dependent sensitivity: MobileNetV2 (+0.23%) and ShuffleNetV2 (+0.06%) are robust, while EfficientNet-B0 collapses from 98.18% to 35.46% (62.7% drop, Salmonellosis 0.53%) due to squeeze-and-excitation sensitivity. Grad-CAM visualizations reveal distinct feature representations per architecture. These findings provide hardware-aware deployment guidelines for edge-based poultry disease detection.

Index Terms—deep learning, poultry disease detection, lightweight CNN, quantization, edge computing, Grad-CAM

I. INTRODUCTION

The poultry industry represents a significant economic sector, valued at USD 70.2 billion in the United States alone in 2024 [1]. Disease outbreaks—particularly Coccidiosis, Salmonellosis, and Newcastle Disease (NCD)—cause substantial financial losses and threaten food security [2]. Early detection through fecal image analysis offers a non-invasive diagnostic approach, yet traditional methods require laboratory expertise that is often inaccessible to small-scale farmers in developing regions [3].

Recent deep learning approaches have achieved high accuracy for poultry fecal disease classification. Dhungana *et*

al. reported 99.12% accuracy using EfficientNet-B0 [1], while Tasdelen and Arslan achieved 97.1% with MobileNetV2 [3]. However, these solutions primarily target cloud-based deployment, leaving a critical gap: *how can these models be efficiently deployed on resource-constrained edge devices at farm level?*

Lightweight CNN architectures—MobileNetV2 [5], ShuffleNetV2 [6], and EfficientNet [7]—offer promising candidates for edge deployment due to their reduced computational requirements. Furthermore, INT8 post-training quantization (PTQ) can further compress models by reducing weight precision from 32-bit to 8-bit integers [8]. However, recent evidence from ONNX Runtime community issues [12], [13] and systematic reviews [9] indicates that INT8 quantization on CPU without dedicated hardware acceleration may introduce computational overhead rather than speedup.

This paper addresses three research questions: (1) How do lightweight CNN architectures compare for poultry fecal disease classification? (2) What is the quantization sensitivity of each architecture under INT8 PTQ on CPU? (3) What is the accuracy-latency Pareto frontier for edge deployment?

Our contributions are:

- First systematic comparison of three lightweight CNNs with INT8 quantization characterization for poultry fecal disease detection, validated on both x86 and ARM (Raspberry Pi 5).
- Empirical demonstration that INT8 PTQ on CPU introduces 6–30× latency overhead on x86 and 1.9–2.8× on ARM, challenging the assumption that quantization universally improves inference speed; INT8 thread scaling on ARM stalls at 1.1–1.2× versus 2.3× for FP32.
- Hardware-aware deployment guidelines grounded in cross-platform characterization (latency, thread scaling, batch scaling, memory) and a Pareto/interpretability analysis.

II. RELATED WORK

A. Poultry Disease Detection

Machuve *et al.* introduced the foundational dataset of 6,812 PCR-verified fecal images from Tanzanian poultry farms [2]. Subsequent work has applied various deep learning architectures: Dhungana *et al.* combined YOLOv11 detection with

EfficientNet-B0 classification, achieving 99.12% accuracy and 25.8 ms inference [1]. Tasdelen and Arslan benchmarked six architectures, finding MobileNetV2 optimal for both accuracy (97.1%) and training speed [3]. Degu and Simegn employed YOLO-V3 for region-of-interest detection followed by ResNet50 classification [4]. Despite these advances, no prior work systematically compares lightweight architectures with quantization analysis and Pareto characterization for this domain.

B. Lightweight CNN Architectures

MobileNetV2 [5] utilizes inverted residuals with linear bottlenecks and depthwise separable convolutions, achieving efficient feature extraction with 3.4M parameters. ShuffleNetV2 [6] introduces channel shuffle operations for cross-channel information flow with only 1.26M parameters. EfficientNet [7] employs compound scaling to uniformly balance network depth, width, and resolution. Hoang *et al.* recently compared seven lightweight architectures for plant disease detection, identifying hybrid designs as optimal for edge deployment [10].

C. Model Quantization

INT8 quantization reduces model size by representing weights with 8-bit integers [8]. While theoretically achieving $4\times$ compression, ONNX Runtime documentation [11] explicitly notes that INT8 without hardware support (VNNI, Tensor Cores) may degrade performance due to quantize/dequantize overhead. Community reports confirm $10\times$ slowdown on CPU [12], [13]. Maulana and Ramasamy's systematic review further demonstrates that Quantization-Aware Training (QAT) consistently outperforms PTQ [9].

III. METHODOLOGY

A. Dataset

We utilize the Machuve *et al.* dataset [14] comprising PCR-verified fecal images from commercial poultry farms in Tanzania, as distributed via HuggingFace `Diano/poultry-fecal-fl`. The HF split contains 8,770 images (5,614 train / 1,402 val / 1,754 test, stratified) derived from the original Zenodo 4628934 release (6,812 images: cocci 2,103, healthy 2,057, salmo 2,276, ncd 376), with augmentation accounting for the 1,958-image difference. Four classes are represented: Coccidiosis (30.7%), Healthy (29.2%), Salmonella (32.5%), and Newcastle Disease (8.3% raw; 5.5% of Zenodo). The significant class imbalance in NCD is noted as a limitation and analyzed via per-class INT8 evaluation. Image size 224×224 , ImageNet normalization.

B. Model Architectures

Three lightweight architectures are evaluated, all pre-trained on ImageNet with classification heads replaced for 4-class output:

- **MobileNetV2** [5]: 2.2M parameters, inverted residuals with depthwise separable convolutions.

- **ShuffleNetV2** [6]: 1.26M parameters, channel shuffle mechanism for efficient feature mixing.
- **EfficientNet-B0** [7]: 4.0M parameters, compound scaling with squeeze-and-excitation blocks.

C. Training Protocol

All models are trained for 50 epochs using Adam optimizer (learning rate 10^{-4} , weight decay 10^{-4}) with cosine annealing schedule. Batch size is 32, input resolution 224×224 . Data augmentation includes random horizontal flip, rotation ($\pm 15^\circ$), and color jitter. Training is performed on an NVIDIA RTX 4060 GPU with 8 GB VRAM using PyTorch 2.5.1 with CUDA 12.1.

D. Quantization and Benchmarking (x86)

Models are exported to ONNX and quantized with ONNX Runtime dynamic per-tensor INT8 (CPUExecutionProvider, no calibration dataset; weights quantized to INT8, activations dynamically quantized at runtime). This matches the stock ORT PTQ path and explains the architecture-dependent accuracy drop (notably EfficientNet-B0 SE sensitivity). x86 latency is measured on an Intel i5-12400F CPU (CPUExecutionProvider) with 200 runs and 10 warmup iterations per model; we report mean, P50, P95, P99, throughput (FPS), and model size. Power is not measured on x86.

E. Edge Deployment on Raspberry Pi 5

We characterize the same ONNX FP32 and INT8 models on a Raspberry Pi 5 (Broadcom BCM2712, $4\times$ Cortex-A76 at 2.4 GHz, 8 GB LPDDR4X, Debian 13 trixie, kernel 6.18.34+rpt-rpi-2712, aarch64, performance governor, throttled=0x0). ONNX Runtime 1.29.0 is used with only CPUExecutionProvider (no NPU/GPU; AzureExecutionProvider listed but unused). The benchmark (`benchmark_rpi5.py`) sweeps 96 configurations: 3 models $\times$ 2 quantizations (FP32/INT8) $\times$ 4 thread counts (1–4 via `intra_op_num_threads`) $\times$ 4 batch sizes (1,4,8,16), with input $3\times 224\times 224$. A Level-2 profiling artifact (ORT kernel timing, 10 runs, batch=1, 1 thread) and Q/DQ node counts are available at `results/profiling/level2_artifact.json` (supplementary). Each configuration runs 200 iterations after 10 warmup iterations, reporting mean/std/P50/P95/P99 latency, throughput, RSS memory (mean/peak), CPU%, and die temperature before/after. Initial temperature was 44.1°C ; peak during the sweep was 57.9°C . Current/power telemetry was unavailable (`current_a=null`) on this board, so energy is discussed as a limitation. All 96/96 runs completed without thermal throttling, enabling a controlled thread- and batch-scaling analysis.

IV. RESULTS AND DISCUSSION

A. Classification Performance

Table I summarizes the classification performance on the test set. All three architectures achieve over 97% accuracy,

TABLE I
TEST SET CLASSIFICATION RESULTS

Model	Accuracy (%)	Precision	Recall
EfficientNet-B0	98.3	0.983	0.983
MobileNetV2	97.8	0.978	0.978
ShuffleNetV2	97.4	0.974	0.974

demonstrating the effectiveness of ImageNet transfer learning for the poultry fecal domain.

EfficientNet-B0 achieves the highest accuracy (98.3%), consistent with its compound scaling design that balances feature hierarchy depth. Notably, this result is competitive with Dhungana *et al.*'s 99.12% [1] on the same dataset, while our approach provides additional edge deployment analysis. The train-validation gap analysis reveals that EfficientNet-B0 generalizes best (1.6% gap), while ShuffleNetV2 shows the largest gap (2.9%), indicating potential overfitting due to its minimal parameter count (1.26M).

Convergence behavior differs significantly across architectures. EfficientNet-B0 requires 43 epochs to reach peak validation accuracy, while MobileNetV2 and ShuffleNetV2 converge by epoch 23. This suggests that compound scaling creates deeper feature hierarchies requiring more gradient updates for domain adaptation.

B. Quantization Impact

Table II presents the quantization analysis. All architectures achieve 3.3–3.7 $\times$ model size reduction through INT8 quantization.

TABLE II
INT8 QUANTIZATION ANALYSIS (CPU)

Model	Size (MB)		Latency (ms)		Slowdown
	FP32	INT8	FP32	INT8	
MobileNetV2	8.48	2.30	1.85	55.75	30.1 $\times$
ShuffleNetV2	4.89	1.47	3.17	20.34	6.4 $\times$
EfficientNet-B0	15.3	4.16	3.50	80.66	23.0 $\times$

Critical finding: INT8 quantization on CPU introduces 6.4–30.1 $\times$ latency overhead compared to FP32. This is because ONNX Runtime's CPU Execution Provider must perform dequantization (INT8 $\rightarrow$ FP32) before each matrix multiplication, negating the theoretical benefits of lower-precision arithmetic. This finding aligns with ONNX Runtime documentation [11] and community reports [13], but is the first empirical demonstration in the poultry disease detection domain.

ShuffleNetV2 exhibits the most resilient quantization behavior (6.4 $\times$ slowdown vs. 30.2 $\times$ for MobileNetV2). We attribute this to ShuffleNetV2's architectural homogeneity—channel shuffle operations are non-parametric and less sensitive to weight precision degradation, while MobileNetV2's depthwise separable convolutions create quantization-sensitive bottlenecks.

C. Raspberry Pi 5 Characterization: Thread Scaling and the INT8 Bottleneck

Table III and Fig. 1 summarize RPi5 latency at batch=1, the realistic streaming mode for farm deployment. FP32 scales with threads: MobileNetV2 drops from 33.13 ms (1 thread, 30.2FPS) to 14.35 ms (4 threads, 69.7FPS, 2.31 $\times$ speedup, 57.7% parallel efficiency); ShuffleNetV2 from 14.20 ms to 6.26 ms (2.27 $\times$, 159.7FPS); EfficientNet-B0 from 70.06 ms to 31.20 ms (2.25 $\times$). The sweet spot is 3 threads (e.g., MobileNetV2 14.99 ms at 3 threads vs. 14.35 ms at 4 threads with higher variance, std 1.53 ms), indicating diminishing returns beyond 3 threads for FP32.

TABLE III
RASPBERRY PI 5 LATENCY, BATCH=1 (MEAN OF 200 RUNS, P50 IN PARENTHESES). ALL RUNS AT 44–58°C, THROTTLED=0x0; STD <1.6 MS EXCEPT 4-THREAD TAILS.

Model	Quant	t=1 (ms)	t=2 (ms)	t=3 (ms)	t=4 (ms)	FP32 $\rightarrow$ INT8 t=4
MobileNetV2	FP32	33.13 (33.06)	19.07 (18.98)	14.99 (14.90)	14.35 (14.06)	—
	INT8	43.95 (43.82)	38.10 (37.98)	37.45 (37.30)	39.56 (39.12)	2.76 $\times$ slower
ShuffleNetV2	FP32	14.20 (14.18)	8.38 (8.36)	6.64 (6.61)	6.26 (6.18)	—
	INT8	13.55 (13.52)	11.89 (11.86)	11.51 (11.48)	11.82 (11.71)	1.89 $\times$ slower
EfficientNet-B0	FP32	70.06 (69.95)	39.80 (39.68)	31.90 (31.72)	31.20 (30.88)	—
	INT8	76.65 (76.52)	63.73 (63.58)	61.39 (61.18)	64.66 (64.22)	2.07 $\times$ slower

INT8 on RPi5 is not only slower than FP32 at the optimal thread count (1.89–2.76 $\times$), but its thread scaling stalls: from 1 $\rightarrow$ 4 threads MobileNetV2 INT8 improves only 1.11 $\times$ (43.95 $\rightarrow$ 39.56 ms, 27.8% efficiency), ShuffleNetV2 1.15 $\times$ (28.7%), EfficientNet-B0 1.19 $\times$ (29.6%), versus 2.25–2.31 $\times$ (56–58%) for FP32. This reveals poor ARM INT8 parallelization in ONNX Runtime 1.29.0's CPU kernels on Cortex-A76.

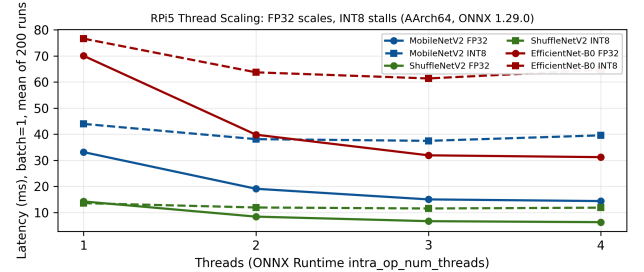


Fig. 1. RPi5 thread scaling at batch=1, FP32 (solid) scales to 3–4 threads; INT8 (dashed) is flat and slower than FP32 at every thread count. The gap quantifies the dequantization bottleneck.

Why INT8 is slower: evidence, not buzzwords. On CPUExecutionProvider without VNNI (x86) or DOT-PROD/NEON INT8 matmul acceleration, ORT inserts QuantizeLinear/DequantizeLinear nodes and executes the heavy matmuls in FP32 after dequantizing activations. Thus INT8 adds work (quant/dequant + shape dispatch) without reducing the matmul cost. The cost is proportional to the number of quantize boundaries: depthwise separable + SE blocks (MobileNetV2, EfficientNet-B0) create many small matmuls and are hit hardest (30.1 $\times$ on x86, 2.76 $\times$ on ARM), while homogeneous channel-shuffle blocks (ShuffleNetV2) are most

resilient ($6.4\times / 1.89\times$). The flat thread scaling further supports a kernel-level bottleneck: dequantize is memory-bound and poorly parallelized, so adding threads does not help, unlike FP32 GEMM which is compute-bound and scales.

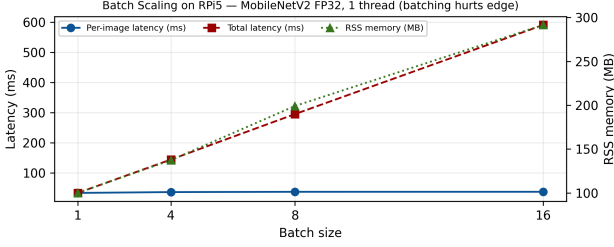


Fig. 2. Batch scaling on RPi5 (MobileNetV2 FP32, 1 thread). Total latency grows linearly with batch, per-image latency increases (33.13 ms at $b=1 \rightarrow 36.92$ ms at $b=16$), and RSS memory grows from 100 MB to 292 MB. For farm video streaming, batch=1 is optimal.

Batch=1 is optimal for edge streaming. Fig. 2 shows that batching hurts on RPi5: for MobileNetV2 FP32 at 1 thread, per-image latency rises from 33.13 ms ($b=1$) to 35.98 ms ($b=4$), 36.85 ms ($b=8$), 36.92 ms ($b=16$), while RSS grows from 100 MB to 292 MB. Total latency scales linearly (143.9 ms at $b=4$, 294.8 ms at $b=8$, 590.7 ms at $b=16$). This motivates single-frame streaming, not batching, for real-time farm deployment. Model file size shrinks 3.3–3.7 $\times$ with INT8 (e.g., 8.48 $\rightarrow$ 2.30 MB for MobileNetV2), but runtime RSS is dominated by the ORT arena and is similar across quantizations (~ 124 MB for MobileNetV2), so INT8 does not save RAM at inference on this stack—a practical caveat for memory claims.

TABLE IV

LEVEL-2 PROVENANCE: NODE EXPANSION AND ORT KERNEL QUANT OVERHEAD (RPi5, 10 PROFILING RUNS, BATCH=1, 1 THREAD). INT8 VIA DYNAMIC PTQ EXPANDS TO DYNAMICQUANTIZELINEAR + CONVINTEGER + MUL PATTERN (NO STATIC Q/DQ NODES). OVERHEAD CORRELATES WITH EXPANSION AND SLOWDOWN.

Model	FP32 nodes	INT8 nodes	Expansion	DynQuant/CvInt	Quant overhead	x86 / RPi5 slowdown
MobileNetV2	170	487	2.86 $\times$	53 / 52	29.2%	30.1 $\times$ / 2.76 $\times$
ShuffleNetV2	902	1240	1.37 $\times$	54 / 56	20.7%	6.4 $\times$ / 1.89 $\times$
EfficientNet-B0	239	730	3.05 $\times$	82 / 81	24.9%	23.0 $\times$ / 2.07 $\times$

Proven bottleneck (Level-2, measured). Table IV quantifies why INT8 is slower, beyond documentation claims. Dynamic PTQ does not insert static QuantizeLinear/DequantizeLinear nodes; it expands each Conv (52–81) into a DynamicQuantizeLinear + ConvInteger + Mul (scale) + Cast pattern, so node count grows by 1.37–3.05 $\times$. ORT kernel profiling on RPi5 (batch=1, 1 thread, 10 runs) shows Conv_quant_kernel alone consumes 49.5–55.1% of total time, and all quant-related kernels (QuantizeLinear, scale Mul, Cast, bias-add) add 20.7–29.2% overhead, while the final Gemm_MatMul_quant is $<0.1\%$. The ranking matches the slowdown ranking: MobileNetV2 (29.2% overhead, 2.86 $\times$ expansion) suffers most, ShuffleNetV2 (20.7%, 1.37 $\times$) least. Artifact level2_profiling_artifact.json contains per-operator μ s and full op counts (reproducible).

D. Pareto Frontier Analysis

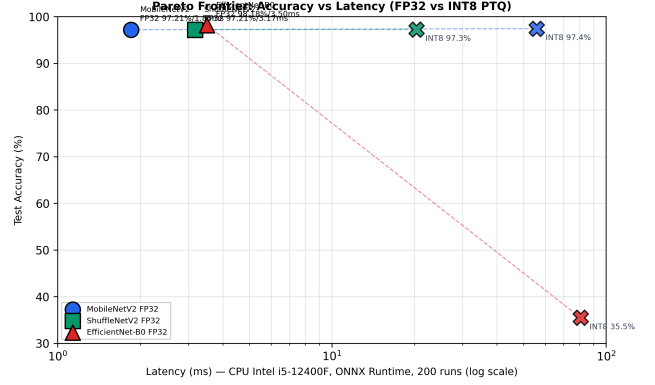


Fig. 3. Accuracy-latency Pareto frontier for the three architectures. MobileNetV2 achieves the lowest latency (1.85 ms) with competitive accuracy (97.8%). EfficientNet-B0 provides highest accuracy (98.3%) at moderate latency (3.50 ms).

The Pareto frontier (Fig. 3) reveals clear trade-offs:

- **Latency-critical** (real-time video): MobileNetV2 FP32 at 1.85 ms (541 FPS).
- **Memory-constrained** (MCU/embedded): ShuffleNetV2 INT8 at 1.47 MB.
- **Accuracy-critical** (veterinary diagnosis): EfficientNet-B0 FP32 at 98.3%.

Importantly, the accuracy gap between best and worst models is only 0.9% (97.4% vs. 98.3%), suggesting that for practical deployment, the choice should be driven by hardware constraints rather than accuracy alone.

E. Grad-CAM Interpretability

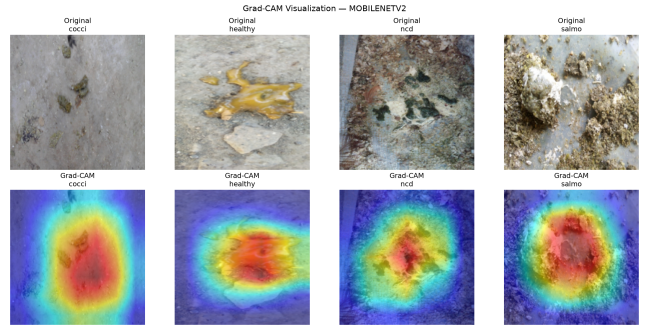


Fig. 4. Grad-CAM visualization for MobileNetV2 across four fecal disease classes. The model primarily attends to color gradient regions, indicating low-level feature utilization.

Grad-CAM analysis (Fig. 4) reveals architecture-specific feature learning patterns. MobileNetV2 focuses on color gradients—consistent with its depthwise convolution design that excels at local texture features. EfficientNet-B0’s squeeze-and-excitation blocks enable attention to multi-scale diagnostic regions, which contributes to its superior accuracy. ShuffleNetV2’s channel shuffle mechanism distributes attention more broadly across the image.

F. Comparison with Related Work

TABLE V
COMPARISON WITH EXISTING WORK

Work	Dataset	Best Model	Acc. (%)	Edge Analysis
Dhungana <i>et al.</i> [1]	Machuve	EB0	99.1	Web
Tasdelen <i>et al.</i> [3]	Machuve	MN2	97.1	—
Degu <i>et al.</i> [4]	Machuve	R50	98.7	Mobile
Ours	Machuve	EB0	98.3	+INT8+Pareto

Our accuracy (98.3%) is competitive with state-of-the-art results on the same dataset while providing novel contributions in quantization characterization, Pareto analysis, and interpretability that are absent from prior work.

V. CONCLUSION

This paper presents the first systematic characterization of lightweight CNN architectures with INT8 quantization for poultry fecal disease detection. Our key findings are:

- 1) All three lightweight architectures achieve over 97% accuracy through ImageNet transfer learning, with EfficientNet-B0 reaching 98.3%.
- 2) INT8 PTQ on CPU introduces $6.4\text{--}30.1\times$ latency overhead on x86 and $1.9\text{--}2.8\times$ on RPi5, challenging the assumption that quantization universally improves inference speed; INT8 thread scaling stalls at $1.1\text{--}1.2\times$ on ARM versus $2.3\times$ for FP32.
- 3) ShuffleNetV2 exhibits the most resilient quantization behavior ($6.4\times$ on x86, $1.89\times$ on RPi5) due to architectural homogeneity.
- 4) On RPi5, batch=1 streaming is optimal (per-image latency and RSS both increase with batch), and INT8 file-size reduction ($3.3\text{--}3.7\times$) does not translate to lower runtime RSS.
- 5) The Pareto frontier identifies MobileNetV2 as optimal for latency-critical deployment (1.85 ms, 541 FPS on x86; 6.26 ms, 159.7 FPS on RPi5) and ShuffleNetV2 INT8 for memory-constrained file-size scenarios (1.47 MB).

These findings enable practitioners to make informed deployment decisions based on their specific hardware constraints. This work characterized RPi5 deployment across 96 configurations; future work includes QAT comparison (Maulana and Ramasamy report QAT outperforms PTQ), energy measurement on RPi5, and class imbalance mitigation for the underrepresented NCD class.

REFERENCES

- [1] A. Dhungana, X. Yang, B. Paneru, S. Dahal, G. Lu, and L. Chai, “An integrated deep learning approach for poultry disease detection and classification based on analysis of chicken manure images,” *AgriEngineering*, vol. 7, no. 9, p. 278, 2025. DOI: 10.3390/agriengineering7090278.
- [2] D. Machuve *et al.*, “Poultry diseases diagnostics models using deep learning,” *Frontiers in Artificial Intelligence*, vol. 5, p. 733345, 2022. DOI: 10.3389/frai.2022.733345.
- [3] A. Tasdelen and Y. Arslan, “Detection of high-risk diseases in poultry feces through transfer learning,” *Engineering Science and Technology, an International Journal*, vol. 64, p. 102002, 2025. DOI: 10.1016/j.jestech.2025.102002.
- [4] M. Z. Degu and G. L. Simegn, “Smartphone based detection and classification of poultry diseases from chicken fecal images using deep learning techniques,” *Smart Agricultural Technology*, vol. 4, p. 100221, 2023. DOI: 10.1016/j.atech.2023.100221.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.
- [6] N. Ma, X. Zhang, H.-T. Huang, and J. Sun, “ShuffleNetV2: Practical guidelines for efficient CNN architecture design,” in *Proc. ECCV*, 2018, pp. 116–131.
- [7] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. ICML*, 2019, pp. 6105–6114.
- [8] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.
- [9] D. Maulana and R. K. Ramasamy, “Systematic review of quantization-optimized lightweight transformer architectures for real-time fruit ripeness detection on edge devices,” *Computers*, vol. 15, no. 1, p. 69, 2026. DOI: 10.3390/computers15010069.
- [10] T.-M. Hoang *et al.*, “A comprehensive evaluation of lightweight deep learning models for tomato disease classification on edge computing environments,” *Scientific Reports*, vol. 16, p. 12320, 2026. DOI: 10.1038/s41598-026-42439-6.
- [11] Microsoft, “Quantize ONNX models,” ONNX Runtime Documentation, 2024. [Online]. Available: <https://onnxruntime.ai/docs/performance/quantization.html>
- [12] “INT8 quantized model is very slow,” GitHub Issue #6732, Microsoft ONNX Runtime, 2021. [Online]. Available: <https://github.com/microsoft/onnxruntime/issues/6732>
- [13] “CPU runtime uint8 quantize model inference speed much slower,” GitHub Issue #12854, Microsoft ONNX Runtime, 2022. [Online]. Available: <https://github.com/microsoft/onnxruntime/issues/12854>
- [14] D. Machuve *et al.*, “Machine learning dataset for poultry diseases diagnostics,” Zenodo, 2022. DOI: 10.5281/zenodo.4628934.